

Angular Interview Guide

Core, Components Core Concepts • Express & MongoDB • Performance & Scaling • Coding, Debugging & HRamp; Forms • RxJS Core Concepts • Express & MongoDB • Performance & Scaling • Coding, Debugging & HRamp; Auth • Performance Core Concepts • Express & MongoDB • Performance & Scaling • Coding, Debugging & HRamp; TypeScript • Coding, Debugging Core Concepts • Express & MongoDB • Performance & Scaling • Coding, Debugging & HRamp; HR

Angular Core, Components, Lifecycle, Data Binding, Directives, Pipes, DI, Routing & Forms

1. What is Angular?

Definition

Angular is a TypeScript-based framework developed by Google for building scalable, component-based Single Page Applications (SPAs) with built-in support for routing, dependency injection, forms, and HTTP services.

Example

```
ng new employee-management ng serve
```

2. What is SPA?

Definition

A Single Page Application (SPA) loads a single HTML page and dynamically updates its content without reloading the browser, providing a faster user experience.

Example

Angular Router handles navigation without page refresh.

3. What is Angular CLI?

Definition

Angular CLI is the official command-line tool used to create, build, test, generate, and manage Angular applications efficiently.

Example

```
ng generate component home
```

4. What is a Component?

Definition

A Component is the fundamental building block of an Angular application that controls a specific part of the user interface using TypeScript, HTML, and CSS.

Example

```
@Component({...}) export class HomeComponent {}
```

5. What is a Standalone Component?

Definition

A Standalone Component works independently without requiring an NgModule, simplifying Angular application development.

Example

```
@Component({standalone:true})
```

6. What are Lifecycle Hooks?

Definition

Lifecycle Hooks are predefined Angular methods that allow you to execute logic at different stages of a component's lifecycle.

Example

```
ngOnInit(), ngOnChanges(), ngOnDestroy()
```

7. What is ngOnInit()?

Definition

ngOnInit() is called once after Angular initializes the component and is commonly used for API calls and initial data loading.

Example

```
ngOnInit(){ this.loadUsers(); }
```

8. What is ngOnDestroy()?

Definition

ngOnDestroy() is executed before a component is destroyed and is used to clean subscriptions, timers, and event listeners.

Example

```
ngOnDestroy(){ this.destroy$.next(); }
```

9. What is Interpolation?

Definition

Interpolation displays TypeScript values inside HTML templates using double curly braces.

Example

```
{{ user.name }}
```

10. What is Property Binding?

Definition

Property Binding binds a component property to an HTML element property using square brackets.

Example

```
[src]="imageUrl"
```

11. What is Event Binding?

Definition

Event Binding executes a component method when a browser event occurs using parentheses.

Example

```
(click)="save()"
```

12. What is Two-Way Binding?

Definition

Two-way Binding synchronizes data between the component and the view using ngModel.

Example

```
[(ngModel)]="username"
```

13. What are Structural Directives?

Definition

Structural Directives add or remove HTML elements from the DOM based on conditions or collections.

Example

```
*ngIf, *ngFor
```

14. What are Pipes?

Definition

Pipes transform data in templates without changing the original value, making it easier to display formatted data.

Example

```
{{ amount | currency }}
```

15. What is Async Pipe?

Definition

The Async Pipe automatically subscribes to and unsubscribes from Observables or Promises in templates.

Example

```
{{ users$ | async }}
```

16. What is a Service?

Definition

A Service is a reusable class that contains business logic or shared functionality used across multiple components.

Example

```
ng generate service user
```

17. What is Dependency Injection?

Definition

Dependency Injection automatically provides service instances to components, making the application modular, reusable, and easy to test.

Example

```
constructor(private userService: UserService) {}
```

18. What is ViewChild?

Definition

ViewChild is used to access a child component or DOM element after the view has been initialized.

Example

```
@ViewChild('txt') input!: ElementRef;
```

19. What is @Input()?

Definition

@Input() allows a parent component to pass data to a child component through property binding.

Example

```
@Input() user!: User;
```

20. What is @Output()?

Definition

@Output() allows a child component to send data or events back to its parent using EventEmitter.

Example

```
@Output() saved = new EventEmitter();
```

21. What is Routing?

Definition

Routing enables navigation between different components without reloading the entire application.

Example

```
{ path: 'home', component: HomeComponent }
```

22. What is Lazy Loading?

Definition

Lazy Loading loads feature modules or routes only when required, improving the application's initial load performance.

Example

```
loadChildren: () => import('./admin.routes')
```

23. What is Route Guard?

Definition

Route Guards control access to routes by allowing or preventing navigation based on specific conditions.

Example

```
CanActivate
```

24. What are Reactive Forms?

Definition

Reactive Forms are model-driven forms that provide better validation, scalability, and testing support for enterprise applications.

Example

```
this.form = this.fb.group({ name: [''] });
```

25. What is FormGroup?

Definition

FormGroup is a collection of multiple FormControl's used to manage an entire form as a single object.

Example

```
new FormGroup({})
```

26. What is FormControl?

Definition

FormControl represents a single input field and tracks its value, validation state, and user interactions.

Example

```
new FormControl('')
```

27. What is FormArray?

Definition

FormArray manages a dynamic list of FormControl's or FormGroup's, making it ideal for repeatable form sections.

Example

```
this.fb.array([])
```

28. What are Validators?

Definition

Validators are built-in or custom functions used to validate user input before form submission.

Example

```
Validators.required
```

29. patchValue() vs setValue()

Definition

patchValue() updates selected form controls, whereas setValue() updates every control and requires all values to be provided.

Example

```
form.patchValue({name: 'John'});
```

30. Template vs Reactive Forms

Definition

Template Forms are simple and template-driven, whereas Reactive Forms are scalable, testable, and recommended for enterprise applications.

Example

```
Reactive Forms + FormBuilder
```

RxJS, HttpClient, Interceptors, JWT Authentication & Signals

31. What is RxJS?

Definition

RxJS is a reactive programming library used in Angular to work with asynchronous data streams such as HTTP calls, events, and timers.

Example

```
this.http.get('/users').subscribe();
```

32. What is an Observable?

Definition

An Observable is a stream that emits one or more values over time and can be subscribed to by multiple consumers.

Example

```
users$.subscribe(data => console.log(data));
```

33. Observable vs Promise

Definition

Observable can emit multiple values and be cancelled, whereas a Promise resolves only once and cannot be cancelled.

Example

```
from(fetch(url))
```

34. What is Subject?

Definition

A Subject is both an Observable and an Observer, allowing values to be multicast to multiple subscribers.

Example

```
const s = new Subject();
```

35. What is BehaviorSubject?

Definition

BehaviorSubject stores the latest emitted value and immediately sends it to new subscribers.

Example

```
user$ = new BehaviorSubject(null);
```

36. Subject vs BehaviorSubject

Definition

Subject has no initial value, while BehaviorSubject requires an initial value and remembers the latest emission.

Example

```
new BehaviorSubject(0)
```

37. What is switchMap()?

Definition

switchMap() cancels the previous Observable and switches to the latest one, making it ideal for search APIs.

Example

```
search.valueChanges.pipe(switchMap(v => api.search(v)))
```

38. What is mergeMap()?

Definition

mergeMap() subscribes to all inner Observables simultaneously and returns all results.

Example

```
items.pipe(mergeMap(x => api.save(x)))
```

39. What is concatMap()?

Definition

concatMap() executes Observables one after another in sequence.

Example

```
items.pipe(concatMap(x => api.save(x)))
```

40. What is exhaustMap()?

Definition

exhaustMap() ignores new emissions until the current Observable completes.

Example

```
clicks.pipe(exhaustMap(()=>login()))
```

41. What is debounceTime()?

Definition

debounceTime() delays emitted values until no new value is received within the specified time.

Example

```
valueChanges.pipe(debounceTime(300))
```

42. What is takeUntil()?

Definition

takeUntil() automatically unsubscribes when another Observable emits, preventing memory leaks.

Example

```
obs.pipe(takeUntil(this.destroy$))
```

43. What is HttpClient?

Definition

HttpClient is Angular's built-in service for making HTTP requests and handling API communication.

Example

```
this.http.get('/api/users')
```

44. What is an Interceptor?

Definition

An Interceptor intercepts every HTTP request and response to add headers, tokens, or handle errors globally.

Example

```
Authorization: Bearer
```

45. What is JWT?

Definition

JWT (JSON Web Token) is a secure token used to authenticate users between the client and server.

Example

```
Bearer eyJhbGciOi...
```

46. What is Refresh Token?

Definition

A Refresh Token is used to obtain a new access token without asking the user to log in again.

Example

```
POST /refresh-token
```

47. What is catchError()?

Definition

catchError() handles HTTP or Observable errors and returns a fallback Observable.

Example

```
pipe(catchError(err => of([])))
```

48. What is retry()?

Definition

retry() automatically retries failed HTTP requests a specified number of times.

Example

```
pipe(retry(2))
```

49. What are Signals?

Definition

Signals are Angular's reactive state primitives that automatically update the UI when their value changes.

Example

```
count = signal(0);
```

50. Signals vs BehaviorSubject

Definition

Signals are best for local component state, while BehaviorSubject is better for asynchronous shared streams.

Example

```
count.update(v => v + 1)
```

Performance, Change Detection, JavaScript & TypeScript

51. What is Change Detection?

Definition

Change Detection is Angular's mechanism for detecting data changes and automatically updating the view.

Example

```
{{ user.name }} updates when value changes.
```

52. Default vs OnPush

Definition

Default checks all components, whereas OnPush checks only when input references, events, or observables/signals trigger updates.

Example

```
changeDetection: ChangeDetectionStrategy.OnPush
```

53. What is ChangeDetectorRef?

Definition

ChangeDetectorRef allows you to manually trigger or control Angular's change detection.

Example

```
cdr.detectChanges();
```

54. What is trackBy?

Definition

trackBy helps Angular reuse existing DOM elements in ngFor, improving rendering performance.

Example

```
trackById(index,item){ return item.id; }
```

55. What is Lazy Loading?

Definition

Lazy Loading loads feature modules only when required, reducing the initial application load time.

Example

```
loadChildren:()=>import('./admin.routes')
```

56. What is Virtual Scrolling?

Definition

Virtual Scrolling renders only visible items instead of the entire list, improving performance for large datasets.

57. Memory Leak

Definition

A memory leak occurs when subscriptions, timers, or event listeners are not released after a component is destroyed.

Example

```
obs.pipe(takeUntil(this.destroy$))
```

58. What is Closure?

Definition

A closure is a function that remembers and accesses variables from its outer scope even after the outer function has finished.

Example

```
function outer(){let x=1; return ()=>x;}
```

59. What is Hoisting?

Definition

Hoisting is JavaScript's behavior of moving variable and function declarations to the top of their scope before execution.

Example

```
console.log(a); var a=10;
```

60. var vs let vs const

Definition

var is function-scoped, let is block-scoped, and const is block-scoped with immutable binding.

Example

```
const PI=3.14;
```

61. What is Event Loop?

Definition

The Event Loop executes synchronous code first, then processes asynchronous callbacks from the task queues.

Example

```
setTimeout(()=>console.log('Done'));
```

62. Promise vs async/await

Definition

async/await provides a cleaner syntax for working with Promises while maintaining asynchronous behavior.

Example

```
const users = await api.getUsers();
```

63. What is Interface?

Definition

An Interface defines the structure that an object must follow in TypeScript.

Example

```
interface User { id:number; name:string; }
```

64. What are Generics?

Definition

Generics allow reusable classes, methods, and interfaces to work with different data types safely.

Example

```
function getData(v:T):T{return v;}
```

65. What is Enum?

Definition

An Enum is a TypeScript feature that defines a set of named constants.

Example

```
enum Role {Admin, User}
```

Coding Questions, Debugging Scenarios, Project Discussion & HR

66. CRUD Application

Definition

CRUD is the process of Creating, Reading, Updating and Deleting records using Angular forms and HttpClient.

Example

```
this.http.post('/users', user)
```

67. Search with debounceTime

Definition

Use debounceTime() and switchMap() to reduce unnecessary API calls while the user types.

Example

```
search.valueChanges.pipe(debounceTime(300), switchMap(v=>api.search(v)))
```

68. Pagination

Definition

Pagination divides large datasets into smaller pages to improve performance and user experience.

Example

```
GET /users?page=1&size;=10
```

69. Infinite Scroll

Definition

Infinite Scroll loads additional records automatically when the user reaches the bottom of the page.

Example

```
(scrolled)="loadMore()"
```

70. File Upload

Definition

Use FormData with HttpClient to upload files to the server.

Example

```
const fd=new FormData(); fd.append('file',file);
```

71. API Called Twice

Definition

Duplicate API calls usually occur because of multiple subscriptions, duplicate events, or repeated component initialization.

Example

```
this.api.getUsers().pipe(shareReplay(1))
```

72. Memory Leak

Definition

Memory leaks occur when subscriptions or listeners remain active after a component is destroyed.

Example

```
obs.pipe(takeUntil(this.destroy$))
```

73. Shared Data Between Components

Definition

Share data using a service with BehaviorSubject or Signals.

Example

```
user$.next(user)
```

74. Authentication Flow

Definition

User logs in, receives a JWT, interceptor attaches the token, and protected APIs validate it.

Example

```
Authorization: Bearer
```

75. Error Handling

Definition

Handle HTTP errors centrally using an interceptor or catchError().

Example

```
pipe(catchError(err=>of([])))
```

76. Project Architecture

Definition

Organize code into Core, Shared, Features, Services, Models and Guards for maintainability.

Example

```
src/app/core, shared, features
```

77. Folder Structure

Definition

Separate components, services, models and routing inside each feature module.

Example

```
features/user/user.component.ts
```

78. Explain Your Project

Definition

Describe the project, your role, technologies used, architecture, challenges and optimizations in a structured way.

Example

```
Angular + .NET + SQL Server
```

79. Common HR Question

Definition

Introduce yourself by summarizing your experience, Angular skills, projects and current responsibilities.

Example

```
'I have strong Angular experience building enterprise web applications.'
```

80. Why Should We Hire You?

Definition

Highlight your Angular expertise, problem-solving ability, project ownership and willingness to learn.

Example

```
'I have delivered multiple production Angular applications with focus on performance and clean architecture.'
```